



Gruppo SkyNet
skynetunigroup@gmail.com

Bug Report

Code Guardian

Versione	1.0
Redattori	Samuele Bertin, Sara Ristovic
Verifica	Dario Pirrone
Approvazione	Marco Barbiero

Versioni del documento

Versione	Data	Descrizione	Autore	Verificatore
1.0	2026/09/22	Individuazione e documentazione dei difetti #10-#14.	Sara Ristovic	Dario Pirrone
0.2	2026/09/20	Individuazione e documentazione dei difetti #5-#10.	Sara Ristovic	Dario Pirrone
0.1	2026/09/18	Strutturazione del documento; Individuazione e documentazione tecnica dei primi 4 difetti applicativi.	Samuele Bertin	Dario Pirrone

Indice

1	Introduzione	3
1.1	Obiettivo del documento	3
1.2	Convenzioni del documento	3
2	Riepilogo dei difetti	3
3	Descrizioni dei difetti	4
#1		4
#2		4
#3		4
#4		5
#5		5
#6		5
#7		5
#8		6
#9		6
#10		6
#11		6
#12		7
#13		7
#14		7

1 Introduzione

1.1 Obiettivo del documento

Questo documento raccoglie i difetti applicativi individuati in Code Guardian. Per ciascun difetto vengono riportati: il componente coinvolto, la gravità e una descrizione breve.

1.2 Convenzioni del documento

- **Gravità:** la scala a cinque livelli: **CRITICA**, **ALTA**, **MEDIA**, **BASSA**, **INFORMATIVA**.
- **Stato:** tutti i difetti di questo documento sono, al momento della stesura, nello stato aperto (nessuna correzione ancora applicata).

2 Riepilogo dei difetti

ID	Titolo	Gravità	Componente
#1	Sprint ID vuoto non validato lato backend. Viene ignorato silenziosamente invece di essere rifiutato	BASSA	Backend
#2	Il limite CONTEXT_TOO_LARGE non copre il contenuto recuperato dai tool nell'agente di Sicurezza	MEDIA	Agenti
#3	Branch GitHub orfano se la creazione della Pull Request fallisce a metà	BASSA	Backend
#4	Errore generico UPSTREAM maschera codici applicativi specifici restituiti dal backend	MEDIA	Agenti + Backend (contratto errori interni)
#5	Nessuna politica di conservazione/eliminazione dei dati	BASSA	Backend, MongoDB
#6	Perdita della coda Redis lascia i task bloccati in PENDING o falliti con errore generico	MEDIA	Backend (coda task) + Infra (docker-compose)
#7	Il testo del pop-up per lo sprintID suggerisce un formato errato	MEDIA	Frontend
#8	Repository di grandi dimensioni: liste GitHub incomplete per mancata paginazione	ALTA	Backend (integrazione GitHub)
#9	Repository di grandi dimensioni: albero dei file troncato senza avviso	ALTA	Backend (integrazione GitHub)

continua nella pagina successiva

(segue dalla pagina precedente)

ID	Titolo	Gravità	Componente
#10	Cluster MongoDB Atlas in una region diversa da quella prevista. Rende PrivateLink inutilizzabile e il traffico è su Internet pubblico.	MEDIA	Infrastruttura (AWS/CDK)
#11	Data/ora non formattata (ISO grezzo) nell'intestazione dei PDF esportati	BASSA	Backend (generazione PDF)
#12	L'interfaccia è in italiano invece che in inglese	Informativa	Frontend
#13	Nessun controllo che il progetto SonarQube collegato corrisponda al repository GitHub analizzato	ALTA	Backend (credenziali) + Agenti (integrazione SonarQube)
#14	La cache delle metriche SonarQube è ancorata al commit, ma la richiesta reale a SonarQube non lo è mai	MEDIA	Agenti (integrazione SonarQube)

3 Descrizioni dei difetti

#1

Il frontend ora impedisce di inviare uno Sprint ID vuoto, ma il backend non applica la stessa regola. Il DTO (`SubmitInputDto`) valida solo che `sprintId` sia una stringa, non che non sia vuota, quindi chiunque aggiri il form (client diretto, o una futura regressione del frontend) può ancora inviare una stringa vuota. Se questo accade, il valore vuoto non viene rifiutato né segnalato ma viene trattato come "nessuno Sprint ID fornito" e l'agente prosegue usando tutte le issue chiuse del repository, senza applicare alcun filtro. Il comportamento è silenzioso e diverso da quanto l'utente probabilmente intendeva, senza che appaia alcun errore. Manca quindi una validazione lato server equivalente a quella già presente nel frontend.

#2

Il controllo sulla dimensione massima del contesto non viene applicato allo stesso modo da tutti gli agenti. Per Documentazione e Changelog, che caricano tutto il contenuto necessario in anticipo, il controllo vede l'intero prompt e funziona correttamente. Le operazioni dell'agente Security (OWASP e Policy Scan), invece, possono recuperare ulteriore contenuto durante l'esecuzione tramite strumenti dedicati (tool) e il controllo sulla dimensione avviene una sola volta, solo sul prompt iniziale, prima che questo recupero aggiuntivo abbia luogo. Qualsiasi contenuto raccolto dopo tramite i tool non viene mai ricontrollato, indipendentemente da quanto diventi grande. Il limite protegge quindi realmente Documentazione e Changelog, ma per le operazioni di Sicurezza è solo apparente. Un'analisi potrebbe elaborare un contesto molto più ampio di quanto il limite dichiarato lasci intendere, senza che scatti mai l'errore.

#3

Quando l'app pubblica una proposta aprendo una Pull Request, esegue in sequenza: crea i blob, crea un albero, crea un commit, crea il branch su GitHub, e solo dopo apre la Pull Request. Se quest'ultimo passaggio fallisce per qualsiasi motivo diverso da un 401/403 generico (es. permessi insufficienti solo sulle PR, un errore temporaneo di GitHub, o un timeout) il branch appena creato non viene eliminato. Nessun codice nell'applicazione esegue un rollback o una pulizia in caso di fallimento a metà. Il risultato è un branch "orfano" lasciato permanentemente nel repository, con un commit reale ma senza alcuna Pull Request associata, visibile solo controllando manualmente l'elenco dei branch su GitHub.

#4

Il servizio redis non ha un volume persistente, a differenza di mongodb e minio. Se il container Redis viene rimosso, come dopo un evento infrastrutturale che tocca il datastore della coda mentre ci sono job in corso, tutto lo stato della coda va perso. L'effetto sui task dipende da dove si trovavano in quel momento:

- I task ancora in attesa in coda restano bloccati per sempre con stato "In attesa": il documento resta nel database, ma nessun worker lo riprenderà mai in carico. Nessun errore visibile, nessun modo per l'utente di distinguerlo da un task solo lento.
- Il task che in quel momento era effettivamente in esecuzione fallisce invece in modo visibile, con un errore generico ("UPSTREAM: Connection closed by server") dovuto alla connessione a Redis interrotta a metà.

#5

Nessuna collezione MongoDB (users, tasks, reports, analysiscontexts, servicecredentials, accesslogs) definisce un indice TTL, e non esiste alcun job schedulato (@Cron o simile) nel backend che elimini record vecchi. Le uniche cancellazioni presenti nel codice sono singole e su richiesta esplicita dell'utente (rimozione di una propria credenziale o di un proprio template README). Non esiste alcuna cancellazione automatica o di massa. Di conseguenza i dati di ogni utente, incluse le operazioni eseguite da tutti i membri del team durante il testing, restano nel database a tempo indeterminato. Non è tecnicamente un malfunzionamento (nulla si comporta in modo scorretto) ma è un'assenza completa di funzionalità in questo ambito.

#6

`docker-compose.yml` non assegna un volume persistente al servizio redis, a differenza di mongodb e minio. La coda BullMQ vive solo nel filesystem del container; se questo viene rimosso (es. `docker compose down + up`) mentre un task è ancora in coda, il job va perso ma il documento Task resta PENDING in MongoDB, senza alcun meccanismo di rilevamento o ripristino, e senza errore visibile per l'utente.

#7

Il popup che richiede lo Sprint ID mostra il testo "Inserisci il numero o codice Sprint, con SPRINT-42 anche come placeholder. In realtà, l'unica integrazione attualmente implementata è con GitHub, e il sistema confronta il valore inserito, carattere per carattere, con il **titolo** esatto di una Milestone GitHub già esistente sul repository e non con un codice sprint in stile Jira. Il testo del popup è scritto in modo volutamente generico, perché il progetto è pensato per supportare in futuro sistemi di project management diversi (non solo GitHub), ma così com'è oggi induce l'utente a inserire un formato che il codice attuale non riconoscerà mai.

Si tratta quindi di allineare il testo visibile all'utente a ciò che il sistema effettivamente supporta oggi. Chiarire, finché l'unica integrazione è GitHub, che va inserito il titolo esatto di una Milestone, aggiornando poi questo testo quando/se verranno aggiunte altre integrazioni.

#8

Diverse chiamate all'API di GitHub in `GithubClientService` (issue, repository dell'utente) richiedono solo la prima pagina di risultati (100 elementi) e non seguono la paginazione per recuperare il resto, trattando quindi una lista parziale come se fosse completa. Su repository piccoli o medi il problema è invisibile, perché tutti gli elementi rientrano già nei primi 100.

Riprodotta e confermata sul repository microsoft/PowerToys: il repository ha 34.074 issue chiuse; recuperando solo le prime 100 (le più recenti per data di creazione, ordinamento di default di GitHub), nessuna delle 100 aveva una milestone assegnata. L'agente Changelog, che verifica lo Sprint ID confrontandolo con le milestone trovate tra le issue chiuse, ha quindi rifiutato una milestone realmente esistente sul repository ("PowerToys 0.102") come inesistente.

La stessa causa (liste non paginate) era già stata trovata separatamente per la risoluzione dei branch, ma quel caso specifico è già risolto. `ContextsService` oggi verifica un branch con una chiamata diretta a quel singolo branch, non più tramite una lista, quindi non ne risente più.

#9

A differenza delle liste di issue e repository (bug #8), l'API di GitHub per l'albero dei file non si pagina: restituisce tutto in un'unica risposta fino a un limite interno, e segnala di aver tagliato il risultato tramite un campo dedicato (`truncated: true`). `GithubClientService.getTree()` non controlla mai questo campo. Per un repository con un numero di file molto elevato, la scansione usata per la selezione dello scope e il rilevamento del linguaggio potrebbe quindi risultare incompleta. Mancherebbero dei file o delle directory, senza che l'utente riceva alcun avviso.

#10

L'infrastruttura AWS è progettata per la region eu-south-1 (Milano) — scelta esplicitamente per motivi di data residency, per `config.ts:3`. Il cluster MongoDB Atlas, creato manualmente dall'agenzia, si trova invece in eu-central-1 (Francoforte). AWS PrivateLink funziona solo tra risorse nella stessa region, quindi il Private Endpoint configurato per Atlas non è mai utilizzabile con un cluster in una region diversa.

Come soluzione temporanea, il codice apre esplicitamente il traffico verso Atlas su Internet pubblico, tramite NAT Gateway, per entrambi backend e agenti Python. Il punto più grave è che questa regola viola deliberatamente una garanzia di sicurezza esplicita del progetto - che il security group degli agenti non deve mai avere una regola di egress verso 0.0.0.0/0. Così che si garantisce a livello di rete, non solo nel codice Python, che gli agenti non possano mai raggiungere Internet. La regola temporanea per Atlas la infrange comunque (limitata alla sola porta MongoDB, ma verso qualsiasi indirizzo IP, non solo quelli di Atlas), lasciando un varco di rete reale finché il disallineamento di region non viene risolto.

C'è inoltre un possibile problema di data residency perché i dati dell'applicazione risiedono fisicamente in Germania (Francoforte), non in Italia (Milano) come previsto dalla configurazione dell'infrastruttura.

#11

L'intestazione di ogni PDF esportato include la data di generazione del report, ma il valore viene inserito senza alcuna formattazione. È una stringa ISO 8601 grezza in UTC (es. 2026-09-23T14:32:07.891Z) mai convertita in un formato leggibile né in un fuso orario specifico prima di finire nel PDF. Il sistema non conosce e non trasmette in nessun punto del flusso il fuso orario del browser dell'utente, quindi un adeguamento automatico al fuso dell'utente richiederebbe una modifica più ampia (il frontend dovrebbe comunicare il proprio fuso orario al backend). Una conversione fissa al fuso di Roma, invece, sarebbe immediatamente realizzabile senza aggiungere dipendenze e non servirebbe inventare l'approccio da zero (la data mostrata a schermo nella pagina dei report).

#12

L'interfaccia è in italiano invece che in inglese. Il problema è sistemico: non esiste alcuna libreria di internazionalizzazione (i18n) nel frontend quindi ogni testo dell'interfaccia è scritto direttamente in italiano, come stringa fissa, dentro ai singoli componenti. Una correzione non è quindi una semplice modifica di configurazione o di un file di traduzione, ma richiede di intervenire manualmente su ogni schermata e componente dell'applicazione.

#13

La credenziale SonarQube è collegata all'utente, non al singolo repository. Chi salva una credenziale inserisce una sola volta l'URL dell'istanza, il progetto e il token, e queste coordinate valgono per tutti i repository che quell'utente analizzerà da quel momento in poi. Nessuna parte del sistema verifica che il progetto SonarQube indicato corrisponda davvero al repository GitHub che si sta analizzando in un dato momento. L'unico controllo esistente è che il token sia valido e abbia accesso a quel progetto, non che sia il progetto giusto.

Conseguenza pratica è che un utente che lavora su due repository diversi, ma ha un'unica credenziale SonarQube salvata, otterrebbe le metriche di qualità dello stesso progetto per entrambi. Analizzando il repository B mentre la credenziale punta ancora al progetto SonarQube del repository A, le metriche di A finirebbero silenziosamente nell'analisi di B, senza alcun avviso o errore.

Questo problema, insieme al #14, è il motivo per cui l'integrazione con SonarQube non è mai stata trattata come una funzionalità completa all'interno del progetto. È stata volutamente lasciata come integrazione opzionale e non rifinita, proprio perché mancano i controlli di coerenza necessari a fidarsi pienamente dei dati che restituisce.

#14

Le metriche di qualità recuperate da SonarQube vengono messe in cache insieme al commit a cui, in teoria, si riferiscono, ma questo legame è solo apparente. Nella richiesta effettivamente inviata a SonarQube per ottenere le metriche non viene mai specificato alcun commit. SonarQube restituisce sempre i risultati della sua analisi più recente per quel progetto, qualunque essa sia, senza modo di chiedere "le metriche relative esattamente a questo commit".

Questo crea un falso senso di precisione. Il sistema si comporta come se le metriche fossero specifiche di un commit preciso, ma in realtà riflettono solo l'ultimo stato analizzato da SonarQube, che potrebbe essere più recente o più vecchio del commit che si sta effettivamente documentando. Il caso più a rischio è l'analisi di un commit passato: se SonarQube nel frattempo ha già analizzato versioni successive del codice (o non ha ancora analizzato quella

corrente), le metriche riportate nel documento generato possono riferirsi a uno stato del codice diverso da quello realmente in esame, senza che nulla lo segnali.